ASCURE Defect Lifecycle Audit

Inspection submission → `InspectionItemResult.isDefect=true` → `Defect` creation. Two creation pathways coexist: eager at submission time, and lazy on Dashboard / Operations Board reads. This audit establishes which one is canonical, why the other exists, and what governance consequences follow. Assessment only — no code changes.

HEADLINE — There are TWO mechanisms that create `Defect` rows: an EAGER transactional path at inspection submission (`InspectionsService.submit`, added 2026-05-13), and a LAZY backfill path (`ensureDefectsForAccessibleItems`, added 2026-04-28 when the `Defect` table was first introduced). The lazy path was originally a transition aid; it is now a backstop for legacy `InspectionItemResult.isDefect=true` rows that pre-date the eager path. Because the lazy backstop is scoped to the CURRENT VIEWER, non-ADMIN users (incl. QA Manager) never materialise legacy items they can't see, and the Operations Board can stay empty even when flagged inspection results exist.

1. End-to-end lifecycle trace

1.1 Inspection submission — the EAGER path

`InspectionsService.submit` (`apps/api/src/inspections/inspections.service.ts:~440–488`) performs the transition `completionStatus → SUBMITTED` in a Prisma transaction that **also** creates one `Defect` row per `InspectionItemResult` with `isDefect=true`:

```
const defectCreateData = this.buildDefectCreateData(inspection.itemResults);
const submitInspection = this.prisma.inspection.update({ ... SUBMITTED ... });
if (defectCreateData.length === 0) return submitInspection;
const [submittedInspection] = await this.prisma.$transaction([
  submitInspection,
  this.prisma.defect.createMany({ data: defectCreateData, skipDuplicates: true }),
]);
```

Source helper: `buildDefectCreateData` (`inspections.service.ts:~1417–1437`) maps every flagged result to a new `Defect` with `status=OPEN`, `severity=item.severity ?? MEDIUM`, `lifecycleStatus=DETECTED`. The schema enforces `Defect.inspectionItemResultId @unique` (`schema.prisma:1203`), so `skipDuplicates` guarantees idempotency.

1.2 Lazy backstop — the BACKFILL path

Two near-duplicate implementations of `ensureDefectsForAccessibleItems` exist:

Location	Filter on InspectionItemResult	Access scope applied
`dashboard.service.ts:560–589`	`isDefect=true AND defect IS NULL AND inspection MATCHES accessible scope`	`accessibleInspectionWhere` = tenant + (non-ADMIN) team membership
`defects.service.ts:1426–1459`	`isDefect=true AND inspection MATCHES accessible scope`. Relies on `createMany({skipDuplicates:true})` to skip already-materialised rows.	`inspectionAccessScope` = tenant + (non-ADMIN) team membership (via siteVisit.team)

1.3 Call sites of the lazy backstop

Endpoint	Code path
GET /dashboard	`dashboard.service.ts:36 — at top of getDashboard`
GET /defects (list)	`defects.service.ts:435 — at top of list()`
GET /defects/operations-board	`defects.service.ts:567 — at top of getOperationsBoard`

`InspectionsService.submit` never calls the lazy backstop — it creates the row eagerly in the same transaction. The Site Visits list (`SiteVisitsService.getRollups`) also never calls the lazy backstop and reads from `InspectionItemResult` directly.

2. Why lazy? Was the behaviour intentional?

Git history reconstructs the design intent:

Date	Commit	What it added	Defect creation pathway
2026-04-28	`c757e17`	Add structured checklist results with defect flag	`InspectionItemResult.isDefect` column. No `Defect` table yet.
2026-04-28	`76b2789`	Add persistent defect status workflow	`Defect` table introduced. **Lazy backfill** `ensureDefectsForAccessibleItems` added to bridge from pre-existing flagged items.
2026-04-29	`964af42`	Add dashboard and map workflow	Dashboard module added. It contains its own near-duplicate copy of the lazy materialiser.
2026-05-13	`77f95d9`	Add configurable defect severity system	**EAGER path** introduced: `buildDefectCreateData` + transactional `defect.createMany` inside `submit()`. From this commit forward, new submissions create the Defect row directly.
2026-05-20	`6a54dcb`	Add defect governance accountability and operations board	Operations Board added. Uses the lazy materialiser as a top-of-handler call.

Reconstructed intent: the lazy materialiser was added as a transition aid when the `Defect` table didn't yet exist as a writer in the submission flow. It was useful — for two weeks — to keep the Dashboard / Defect list showing flagged items while the eager pathway was being designed. After the eager pathway shipped on May 13, the lazy backstop became dead weight for any *new* inspection but remained necessary for already-submitted `InspectionItemResult` rows that lacked a `Defect`.

So: was it intentional? The original lazy mechanism was intentional. Its *continued existence in three call sites today* looks like organic accumulation rather than designed steady state. The two near-duplicate copies (in `dashboard.service.ts` and `defects.service.ts`) are a tell.

3. Does Operations Board depend on Dashboard access to materialise defects?

No, not directly. `DefectsService.getOperationsBoard` (`defects.service.ts:563–580`) calls `this.ensureDefectsForAccessibleItems(user)` itself at line 567 before reading. So Operations Board does not need Dashboard to have run first.

But: the materialiser is scoped to the CURRENT VIEWER. If a non-ADMIN with no team membership opens Operations Board, `ensureDefectsForAccessibleItems` finds 0 accessible `InspectionItemResult` rows and creates 0 Defects — even though flagged results exist in the database for that tenant. After that call, the outer `defect.findMany` returns 0 because the Defect row was never created (and even if it had been, the team-scope would drop it).

Therefore Operations Board effectively depends on an ADMIN (or a future QA-actor with cross-team read access) having visited it at least once. That dependency is implicit, not declared. It is the most fragile part of the current model.

4. Can a flagged inspection result exist indefinitely without a Defect row?

Yes. Conditions under which a flagged `InspectionItemResult` remains orphaned:

- Inspection was submitted by a code path that does not call `InspectionsService.submit` (e.g., direct `prisma.inspection.update` in a migration, Studio, or a future endpoint). No eager createMany runs.
- Inspection was submitted before commit `77f95d9` (2026-05-13). Eager path didn't exist then; the lazy backstop is the only path.
- After submission, only non-ADMIN, non-team-member users (e.g., QA Manager with role=MANAGER and no membership in the inspection's team) ever open Dashboard / Operations Board / Defect list. The lazy backstop's viewer-scoped filter returns 0 accessible items and creates nothing.
- The eager transaction at submission throws for some unrelated reason before the `createMany` step. Because both writes are in the same Prisma transaction, the inspection update would roll back too — so submission fails atomically. *This particular case is safe* — but the inspection would remain in DRAFT, not show up as submitted at all.

Consequence of orphaned flagged items:

Surface	Behaviour
Site Visits list — defectsFound column	Counts the orphaned `InspectionItemResult` row. Shows ≥1.
Dashboard defect totals	Counts only `Defect` rows. Shows 0 until ADMIN visits.
Operations Board	Counts only `Defect` rows + nested team scope. Shows 0 for non-ADMIN even after materialisation; shows ≥1 for ADMIN after first ADMIN visit.
QA verify / close authority	`assertCanGovernQa` requires an actor capable of acting on a Defect row. Without the row, QA cannot perform verify / close. **The defect is invisible to governance.**

5. Recommended operational model for ASCURE pilot governance

Two principles drive the recommendation:

- **Single source of truth.** Defect creation belongs in exactly one path — inspection submission. The current two-path model produces the JK6B-style mismatch where the Site Visits page and the Operations Board disagree.
- **Defects are governance artifacts.** The point at which an inspection is submitted is the point at which a governance event exists. From that moment, the defect must be addressable by QA — regardless of who visits a UI page.

5.1 Eager-only model

- `InspectionsService.submit` remains the only writer of `Defect` rows. No other endpoint creates them.
- Eager createMany runs unconditionally in the submission transaction. Idempotent via `@unique inspectionItemResultId`.
- Remove the three call sites of `ensureDefectsForAccessibleItems`; remove both copies of the method. Operations Board and Dashboard read pure `Defect` rows.

- Run a one-time backfill migration at deploy: `INSERT INTO Defect ... SELECT ... FROM InspectionItemResult WHERE isDefect = true AND id NOT IN (SELECT inspectionItemResultId FROM Defect)`. Documented, idempotent, runs once. After the backfill, the lazy path is provably unnecessary.

5.2 Unify the count source

- `SiteVisitsService.getRollups` currently counts `InspectionItemResult` with `isDefect=true`. Switch it to count `Defect` rows (or join via `inspectionItemResult.isDefect=true`) so the same denominator is shown across every surface.
- Acceptable interim: leave the Site Visits rollup using `InspectionItemResult` but require backfill on deploy so the two tables always agree.

5.3 Decouple access scope from data materialisation

- Materialisation should NEVER be viewer-scoped. The current pattern means a non-ADMIN viewer can never materialise items they can't see, leaving them invisible to everyone except a later ADMIN visit. This is a governance defect in itself: it makes the visibility of defects depend on who happens to log in.
- If a lazy path is retained at all, it must use a tenant-wide scope (not the viewer's scope). Then access control is layered on the read side only — where it belongs.

5.4 Pilot-grade implementation order (no code changes proposed here)

- Step 1 — run a one-time backfill SQL/Prisma script against production: every `InspectionItemResult.isDefect=true` without a `Defect` gets a `Defect` row. Backfill is identical to the eager createMany shape. Idempotent.
- Step 2 — switch `SiteVisitsService.getRollups` to read from the `Defect` table. Every surface now agrees on a single denominator.
- Step 3 — remove the three call sites of `ensureDefectsForAccessibleItems` and both helper method copies. Verify that production has zero orphaned flagged items via the backfill report before deploying.
- Step 4 — separately (out of scope here), fix QA visibility: make the four access scopes consult `isQaActor` so QA Managers with the `QA_VALIDATION` capability bypass the team-membership filter on read paths.

6. Bottom line

The lazy `ensureDefectsForAccessibleItems` path is a transition relic that became a backstop. With the eager path in place since 2026-05-13, the lazy path's only job is to clean up legacy flagged items — and it does that imperfectly because it runs in the current viewer's scope rather than the tenant's. A flagged inspection result CAN exist indefinitely without a `Defect` row, and when that happens, governance becomes operationally blind to it. The pilot-correct model is eager-only, with a one-time backfill on deploy and the lazy backstop retired.